LH Nautical — Análise Estratégica de Dados

Diagnóstico Operacional e Recomendações Baseadas em Dados

Analista: Natalia Moraes
Data: Março de 2026
Ferramentas: Python 3, DuckDB, Pandas, Scikit-learn

---

1. Contexto

A LH Nautical é uma empresa do setor náutico que comercializa motores, equipamentos de navegação e itens de ancoragem. Seu modelo de operação combina receitas em moeda local (BRL) com custos atrelados ao dólar (USD), tornando a variação cambial um fator crítico para a rentabilidade.

Apesar de possuir um volume relevante de dados operacionais, a empresa ainda não utiliza essas informações de forma integrada para suportar decisões estratégicas, especialmente em temas como precificação, gestão de clientes e planejamento de demanda.

---

2. Objetivo

O objetivo deste projeto é analisar a base de dados da LH Nautical para identificar problemas operacionais, oportunidades de melhoria e propor ações com impacto direto na performance financeira e na eficiência do negócio.

Especificamente, o projeto busca:

- Avaliar a qualidade e consistência dos dados disponíveis;
- Identificar prejuízos operacionais associados à precificação;
- Mapear o perfil de clientes de alto valor;
- Analisar padrões temporais de vendas;
- Desenvolver modelos analíticos para apoio à decisão, incluindo previsão de demanda e recomendação de produtos.

---

3. Abordagem

A análise foi estruturada de forma incremental, priorizando a construção de confiança nos dados antes da aplicação de qualquer modelo analítico.

O fluxo seguiu cinco etapas principais:

1. **Diagnóstico:** Identificação da estrutura, qualidade e limitações dos dados.
2. **Tratamento:** Padronização, conversão de tipos e remoção de inconsistências.
3. **Análise de negócio:** Avaliação de margem, comportamento de clientes e sazonalidade.
4. **Modelagem:** Aplicação de modelos de previsão e recomendação sobre a base tratada.
5. **Consolidação:** Síntese dos resultados em insights acionáveis e recomendações.

A premissa central deste trabalho é que modelos só são confiáveis quando os dados que os alimentam também são. Todas as decisões técnicas adotadas ao longo do notebook seguem esse princípio.

---

4. Preparação do Ambiente

Foram utilizadas bibliotecas padrão do ecossistema Python para análise de dados, manipulação de estruturas e integração com SQL.

- **pandas:** Manipulação e transformação de dados tabulares.
- **numpy:** Operações numéricas e vetorização.
- **duckdb:** Execução de consultas SQL diretamente sobre DataFrames. A conexão global `conn = duckdb.connect()` é instanciada nesta etapa e reutilizada em todas as análises subsequentes do notebook.
- **requests:** Consumo da API do Banco Central do Brasil (BCB) para obtenção da taxa de câmbio oficial (PTAX).
- **scikit-learn:** Cálculo de similaridade de cosseno para o sistema de recomendação.

> Insight: A combinação de Python com DuckDB permite alternar entre abordagens analíticas e SQL de forma eficiente, reproduzindo um ambiente híbrido comum em pipelines modernos de dados.

In [1]:
```
# ── 4. Preparação do Ambiente ──────────────────────────
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import requests
import duckdb
import unicodedata
from datetime import datetime, timedelta
```

```python
# Configurações de visualização
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.2f}'.format)
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 5)

# Conexão global DuckDB
conn = duckdb.connect()

# Função auxiliar para exibição monetária
def formatar_brl(valor: float) -> str:
    return f"{valor:,.2f}".replace(",", "X").replace(".", ",").replace("X",
```

---

5. Estrutura do Documento

A análise foi estruturada de forma sequencial, evoluindo do diagnóstico inicial dos dados até a geração de insights e aplicações analíticas.

1. Contexto
2. Objetivo
3. Abordagem
4. Preparação do Ambiente
5. Estrutura do Documento

Diagnóstico e Qualidade dos Dados

6. EDA — Análise Exploratória de Dados
7. Padronização da Base de Produtos
8. Estruturação do Histórico de Custos de Importação

Análises de Negócio

9. Análise de Margem e Identificação de Prejuízo por Produto
10. Análise de Clientes com Foco em Fidelização e Rentabilidade
11. Análise Temporal com Calendário Completo

Modelagem Analítica

12. Previsão de Demanda
13. Sistema de Recomendação por Similaridade de Comportamento de Compra

Conclusões e Direcionamentos

14. Conclusões e Recomendações Finais
15. Direcionamento Estratégico

---

6. EDA — Análise Exploratória de Dados

O ponto de partida de qualquer projeto de dados é entender o que existe antes de transformar ou modelar qualquer coisa. Esta seção estabelece o diagnóstico inicial do dataset `vendas_2023_2024.csv` sobre volume, cobertura temporal, qualidade estrutural e distribuição dos dados.

> **Insight principal:** O dataset apresenta boa integridade estrutural, mas três problemas precisam ser tratados antes de qualquer análise: outliers em `total`, formatos de data inconsistentes e ausência de integração com os custos de importação.

6.1 Visão Geral do Dataset

O dataset foi inspecionado para validar volume, cobertura temporal e integridade estrutural antes de qualquer transformação.

- **Volume:** 9.895 linhas, 6 colunas (`id`, `id_client`, `id_product`, `qtd`, `total`, `sale_date`).
- **Cobertura temporal:** 01/01/2023 a 31/12/2024.
- **Valores nulos:** Zero em todas as colunas.
- **Valor mínimo de `total`:** R$ 294,50.
- **Valor máximo de `total`:** R$ 2.222.973,00.
- **Valor médio de `total`:** R$ 263.797,83.

> Insight: A ausência de valores nulos indica boa integridade da base, porém a elevada amplitude da variável `total` sugere a presença de outliers que podem distorcer métricas agregadas sem tratamento prévio.

In [2]:
```
# ── 6.1 Visão Geral do Dataset ──────────────────────────────
df_vendas = pd.read_csv('data/vendas_2023_2024.csv')

# Conversão da coluna de data
df_vendas['sale_date'] = pd.to_datetime(
    df_vendas['sale_date'],
    format='mixed',
    dayfirst=True
)

print("=" * 50)
print("VISÃO GERAL DO DATASET")
print("=" * 50)
print(f"Total de linhas: {df_vendas.shape[0]}")
print(f"Total de colunas: {df_vendas.shape[1]}")
print(f"\nColunas: {list(df_vendas.columns)}")
```

```python
print(f"\nData mínima: {df_vendas['sale_date'].min().date()}")
print(f"Data máxima: {df_vendas['sale_date'].max().date()}")

print("\n" + "=" * 50)
print("ANÁLISE DA COLUNA 'total'")
print("=" * 50)
print(f"Valor mínimo: R$ {formatar_brl(df_vendas['total'].min())}")
print(f"Valor máximo: R$ {formatar_brl(df_vendas['total'].max())}")
print(f"Valor médio: R$ {formatar_brl(df_vendas['total'].mean())}")

print("\n" + "=" * 50)
print("QUALIDADE DOS DADOS")
print("=" * 50)
print("Valores nulos por coluna:")
print(df_vendas.isnull().sum())

display(df_vendas.head(3))
```

```
==================================================
VISÃO GERAL DO DATASET
==================================================
Total de linhas: 9895
Total de colunas: 6

Colunas: ['id', 'id_client', 'id_product', 'qtd', 'total', 'sale_date']

Data mínima: 2023-01-01
Data máxima: 2024-12-31

==================================================
ANÁLISE DA COLUNA 'total'
==================================================
Valor mínimo: R$ 294,50
Valor máximo: R$ 2.222.973,00
Valor médio: R$ 263.797,83

==================================================
QUALIDADE DOS DADOS
==================================================
Valores nulos por coluna:
id            0
id_client     0
id_product    0
qtd           0
total         0
sale_date     0
dtype: int64
```

	id	id_client	id_product	qtd	total	sale_date
0	0	42	105	11	3405.00	2023-10-09
1	1	3	136	9	16873.90	2024-09-15
2	2	25	139	7	9475.30	2024-08-13

6.2 Pontos de Atenção Identificados

A inspeção inicial identificou três pontos críticos que devem ser tratados antes de qualquer análise subsequente.

- **Outliers em total:** O valor máximo (R $2.222.973,00) é aproximadamente 8,4 vezes maior que a média (R$ 263.797,83), indicando a presença de pedidos atípicos que podem distorcer métricas agregadas como ticket médio e faturamento total.
- **Formatos de data inconsistentes:** A coluna sale_date apresenta formatos mistos, exigindo padronização com pd.to_datetime(..., format='mixed') antes de qualquer análise temporal.
- **Nomenclatura em inglês:** Os nomes das colunas indicam origem em sistema externo. O padrão foi mantido ao longo do notebook para garantir consistência na integração com outras bases.

> Insight: A ausência de validações no pipeline de ingestão sugere falta de controles de qualidade na origem, um risco que tende a se amplificar com o crescimento do volume de dados e da complexidade das integrações.

6.3 Diagnóstico de Confiabilidade

O dataset é adequado para análises, desde que os pontos críticos identificados sejam tratados nas etapas seguintes.

- **Outliers:** Devem ser considerados no cálculo de métricas como ticket médio e faturamento, evitando distorções em análises agregadas.
- **Datas:** A padronização da coluna `sale_date` é obrigatória antes de qualquer operação temporal ou integração com outras bases.
- **Integração:** A conversão de moeda é pré-requisito para análises de margem, garantindo a compatibilidade entre vendas registradas em BRL e custos de importação em USD.

> Insight: A base é confiável para análises, porém sua qualidade depende de tratamentos pontuais que idealmente deveriam ser tratados na origem, e não replicados a cada ciclo analítico.

6.4 Consolidação via SQL (DuckDB)

As métricas calculadas em Python foram reproduzidas em SQL utilizando DuckDB como mecanismo de validação cruzada.

- **Método:** Registro dos DataFrames no DuckDB via `conn.register()` e execução de queries SQL diretamente sobre os dados em memória.
- **Resultado:** As métricas obtidas apresentaram valores idênticos aos calculados em Python, validando a consistência dos resultados.
- **Conexão global:** A conexão `conn = duckdb.connect()` foi instanciada nesta etapa e reutilizada ao longo do notebook, garantindo eficiência e padronização na execução das queries.

> Insight: A consistência entre Python e SQL valida a integridade da camada de ingestão e estabelece uma base confiável para as etapas subsequentes de tratamento e modelagem.

In [3]:
```
# ── 6.4 Consolidação via SQL (DuckDB) ──────────────────
conn.register('df_vendas', df_vendas)

query_eda = """
SELECT
    COUNT(*) AS total_linhas,
    MIN(sale_date) AS data_minima,
    MAX(sale_date) AS data_maxima,
    MIN(total) AS total_minimo,
    MAX(total) AS total_maximo,
    AVG(total) AS total_medio
FROM df_vendas
"""

resultado_eda = conn.execute(query_eda).df()
display(resultado_eda)
```

	total_linhas	data_minima	data_maxima	total_minimo	total_maximo	total_medio
0	9895	2023-01-01	2024-12-31	294.50	2222973.00	263797.83

---

7. Padronização da Base de Produtos

O arquivo `data/produtos_raw.csv` apresentava inconsistências que comprometiam a confiabilidade das análises, incluindo categorias com grafias variadas, preços em formato textual e registros duplicados.

Esta seção documenta o processo de normalização aplicado para garantir representação única dos produtos, tipagem correta dos campos e consistência na classificação das categorias.

> Insight: Sem essa etapa, análises por categoria seriam distorcidas por inconsistências de grafia, métricas de preço seriam inviabilizadas e o sistema de recomendação produziria resultados incorretos.

7.1 Diagnóstico Inicial da Base de Produtos

O arquivo `data/produtos_raw.csv` foi inspecionado para avaliar a consistência estrutural e identificar inconsistências antes de qualquer transformação.

- **Estrutura:** 157 linhas, 4 colunas (`name` , `price` , `code` , `actual_category`).
- **Tipos:** A coluna `price` está em formato textual (`str`) e `actual_category` apresenta múltiplas variações de grafia para as três categorias válidas.
- **Valores nulos:** Zero em todas as colunas.

> Insight: A ausência de valores nulos não garante qualidade dos dados — inconsistências em categorias e formatação textual de preços tornam a base inadequada para análise sem tratamento prévio.

In [4]:
```
# ── 7.1 Diagnóstico inicial da base de produtos ──────────
df_produtos = pd.read_csv('data/produtos_raw.csv')

print(f"Shape: {df_produtos.shape}")
print(f"\nColunas encontradas: {list(df_produtos.columns)}")
print(f"\nTipos:\n{df_produtos.dtypes}")
print(f"\nNulos:\n{df_produtos.isnull().sum()}")

display(df_produtos.head(5))
```

```
Shape: (157, 4)

Colunas encontradas: ['name', 'price', 'code', 'actual_category']

Tipos:
name                 str
price                str
code               int64
actual_category      str
dtype: object

Nulos:
name               0
price              0
code               0
actual_category    0
dtype: int64
```

	name	price	code	actual_category
0	Transponder AIS Maré Magnum	R$ 33122.52	1	ELETRONICOS
1	Transponder Furuno Marlin	R$ 13998.15	2	ELETRONICOS
2	Radar Furuno Pulse Leviathan	R$ 9024.19	3	E L E T R Ô N I C O S
3	Rádio AIS Hydro Tidal Zen	R$ 3381.88	4	Eletrunicos
4	Piloto Automático Furuno Storm	R$ 23669.01	5	Eletronicoz

7.2 Tratamento e Padronização

Três transformações foram aplicadas de forma sequencial para garantir a consistência da base.

- **Padronização de categorias:** Normalização de texto via `unicodedata`, combinada com correspondência parcial por substrings, permitindo capturar variações como `Eletronicoz` e `E L E T R Ô N I C O S` sem dependência de um dicionário fixo.
- **Conversão de preços:** Remoção do prefixo `R$` e dos separadores no padrão brasileiro, seguida da conversão para `float64`, viabilizando operações analíticas.
- **Remoção de duplicatas:** Aplicação de `drop_duplicates()` após a padronização, revelando 7 registros duplicados que anteriormente não eram identificados devido a diferenças de grafia.

> Insight: A ordem das transformações é deliberada — a padronização de categorias antes da deduplicação garante que registros inicialmente distintos por grafia sejam corretamente identificados como duplicatas.

In [5]:
```
# —— 7.2 Tratamento e Padronização (Carregamento e inspeção inicial de 'data
import pandas as pd
import unicodedata

df_produtos = pd.read_csv('data/produtos_raw.csv')

print(f"Shape: {df_produtos.shape}")
print(f"\nColunas encontradas: {list(df_produtos.columns)}")
print(f"\nTipos:\n{df_produtos.dtypes}")
print(f"\nNulos:\n{df_produtos.isnull().sum()}")

display(df_produtos.head(5))


# —— 7 Tratamento e Padronização (Padronização de categorias) ————————
def normalizar_texto(texto):
    texto = str(texto).strip().lower()
    texto = unicodedata.normalize('NFKD', texto)
    texto = ''.join(c for c in texto if not unicodedata.combining(c))
    texto = texto.replace(' ', '')
```

```python
    return texto

def padronizar_categoria(cat):
    cat = normalizar_texto(cat)

    if any(x in cat for x in ['eletron', 'eletr', 'eletrun', 'eletro']):
        return 'eletrônicos'
    elif any(x in cat for x in ['propul', 'propus', 'propuc', 'prop']):
        return 'propulsão'
    elif any(x in cat for x in ['ancor', 'ankor', 'encor']):
        return 'ancoragem'
    else:
        return cat

df_produtos['actual_category'] = df_produtos['actual_category'].apply(padror

print("Categorias após padronização:")
print(df_produtos['actual_category'].value_counts())


# ── 7 Tratamento e Padronização (Conversão de price para numérico) ────────
df_produtos['price'] = (
    df_produtos['price']
    .str.replace('R$', '', regex=False)
    .str.strip()
    .astype(float)
)

print(f"\nTipo da coluna price: {df_produtos['price'].dtype}")


# ── 7 Tratamento e Padronização (Remoção de duplicatas) ──────────
total_antes = len(df_produtos)

df_produtos = df_produtos.drop_duplicates()

total_depois = len(df_produtos)
duplicatas_removidas = total_antes - total_depois

print(f"\nRegistros antes: {total_antes}")
print(f"Registros depois: {total_depois}")
print(f"Duplicatas removidas: {duplicatas_removidas}")
print(f"\nCategorias finais: {sorted(df_produtos['actual_category'].unique()

display(df_produtos.head(5))


# ── 7 Validação Pós-tratamento ──────────
print("Categorias padronizadas finais:", sorted(df_produtos['actual_category
print("Quantidade de valores nulos em price:", df_produtos['price'].isnull()
print("Total de produtos únicos após deduplicação:", len(df_produtos))
```

```
Shape: (157, 4)

Colunas encontradas: ['name', 'price', 'code', 'actual_category']

Tipos:
name                 str
price                str
code               int64
actual_category      str
dtype: object

Nulos:
name               0
price              0
code               0
actual_category    0
dtype: int64
```

	name	price	code	actual_category
0	Transponder AIS Maré Magnum	R$ 33122.52	1	ELETRONICOS
1	Transponder Furuno Marlin	R$ 13998.15	2	ELETRONICOS
2	Radar Furuno Pulse Leviathan	R$ 9024.19	3	E L E T R Ô N I C O S
3	Rádio AIS Hydro Tidal Zen	R$ 3381.88	4	Eletrunicos
4	Piloto Automático Furuno Storm	R$ 23669.01	5	Eletronicoz

```
Categorias após padronização:
actual_category
propulsão      53
ancoragem      53
eletrônicos    51
Name: count, dtype: int64

Tipo da coluna price: float64

Registros antes: 157
Registros depois: 150
Duplicatas removidas: 7

Categorias finais: ['ancoragem', 'eletrônicos', 'propulsão']
```

	name	price	code	actual_category
0	Transponder AIS Maré Magnum	33122.52	1	eletrônicos
1	Transponder Furuno Marlin	13998.15	2	eletrônicos
2	Radar Furuno Pulse Leviathan	9024.19	3	eletrônicos
3	Rádio AIS Hydro Tidal Zen	3381.88	4	eletrônicos
4	Piloto Automático Furuno Storm	23669.01	5	eletrônicos

```
Categorias padronizadas finais: ['ancoragem', 'eletrônicos', 'propulsão']
Quantidade de valores nulos em price: 0
Total de produtos únicos após deduplicação: 150
```

7.3 Validação Pós-tratamento

Esta etapa valida a consistência da base após as transformações aplicadas.

- **Categorias:** Três categorias padronizadas (`eletrônicos` , `propulsão` , `ancoragem`), sem variações residuais.
- **Preços:** Ausência de valores nulos na coluna `price` após a conversão para `float64` .
- **Volume:** 150 produtos únicos após a remoção de 7 registros duplicados.

> Insight: A redução de 157 para 150 produtos representa aproximadamente 4,5% da base original. Sem essa etapa, métricas agregadas por produto seriam infladas e o sistema de recomendação produziria resultados incorretos devido à duplicidade de registros.

In [6]:
```
# ── 7.3 Validação Pós-tratamento ──────────
print("Categorias padronizadas finais:", sorted(df_produtos['actual_category
print("Quantidade de valores nulos em price:", df_produtos['price'].isnull()
print("Total de produtos únicos após deduplicação:", len(df_produtos))
```

```
Categorias padronizadas finais: ['ancoragem', 'eletrônicos', 'propulsão']
Quantidade de valores nulos em price: 0
Total de produtos únicos após deduplicação: 150
```

7.4 Interpretação — Decisões de Limpeza dos Produtos

As transformações aplicadas seguem uma lógica técnica deliberada, orientada à robustez e à consistência da base.

- **Padronização por substrings:** A abordagem por correspondência parcial foi escolhida por sua resiliência a variações futuras. A normalização remove acentos, converte para minúsculas e elimina espaços antes da correspondência, capturando inclusive casos extremos como `E L E T R Ô N I C O S` e `Eletronicoz` . Novas grafias que contenham `eletron` , `propul` ou `ancor` serão corretamente classificadas sem necessidade de manutenção manual.
- **Conversão de `price` :** A presença de valores em formato textual é um indicativo típico de dados originados em planilhas. A conversão para `float64` é pré-requisito para operações aritméticas e, em um pipeline produtivo, deve ocorrer na camada de ingestão.
- **Ordem da deduplicação:** A remoção de duplicatas após a padronização garante que registros inicialmente distintos por grafia sejam corretamente identificados e eliminados, evitando falsos negativos no processo de deduplicação.

> Insight: As inconsistências identificadas são sintomas de um processo de cadastro de produtos sem validações na origem. Em escala, esse

> problema compromete a confiabilidade de análises baseadas em categorias e preços.

---

8. Estruturação do Histórico de Custos de Importação

O arquivo `custos_importacao.json` armazena o histórico de preços de compra de cada produto em formato aninhado, no qual cada produto contém uma lista de registros ao longo do tempo.

Esse formato não é compatível com análises tabulares nem com a integração direta com outras bases relacionais.

> Insight: A normalização desse arquivo é pré-requisito para análises de margem, pois permite identificar o custo vigente de cada produto na data de cada transação.

8.1 Inspeção da Estrutura dos Dados

O arquivo foi inspecionado para validar a estrutura da raiz, a quantidade de produtos e o formato interno de cada registro antes da definição da estratégia de normalização.

- **Estrutura raiz:** Lista com 150 produtos.
- **Campo aninhado:** `historic_data` com múltiplas entradas de `start_date` e `usd_price` por produto.
- **Incompatibilidade:** O formato hierárquico não permite operações relacionais diretas, como joins e agregações.

> Insight: A estrutura aninhada é adequada para armazenamento e transporte, mas exige normalização para viabilizar integração com bases relacionais e análises temporais.

In [7]:
```
# ── 8.1 Inspeção da Estrutura dos Dados ──────────
with open('data/custos_importacao.json', 'r', encoding='utf-8') as f:
    raw = json.load(f)

# Verificar tipo e estrutura
print(f"Tipo do JSON: {type(raw)}")
if isinstance(raw, list):
    print(f"Total de itens na lista: {len(raw)}")
    print(f"\nPrimeiro item:")
    print(json.dumps(raw[0], indent=2, ensure_ascii=False))
elif isinstance(raw, dict):
    print(f"Chaves do dicionário: {list(raw.keys())}")
    primeira_chave = list(raw.keys())[0]
    print(f"\nPrimeiro item ({primeira_chave}):")
    print(json.dumps(raw[primeira_chave], indent=2, ensure_ascii=False))
```

Tipo do JSON: <class 'list'>
Total de itens na lista: 150

Primeiro item:

```
{
  "product_id": 1,
  "product_name": "Transponder AIS Maré Magnum",
  "category": "eletrônicos",
  "historic_data": [
    {
      "start_date": "10/08/2016",
      "usd_price": 10583.63
    },
    {
      "start_date": "15/06/2018",
      "usd_price": 8778.36
    },
    {
      "start_date": "25/09/2018",
      "usd_price": 8023.87
    },
    {
      "start_date": "19/03/2019",
      "usd_price": 8772.78
    },
    {
      "start_date": "17/01/2020",
      "usd_price": 7918.18
    },
    {
      "start_date": "17/06/2020",
      "usd_price": 6310.01
    },
    {
      "start_date": "02/07/2021",
      "usd_price": 6586.7
    },
    {
      "start_date": "16/05/2022",
      "usd_price": 6538.2
    },
    {
      "start_date": "28/02/2023",
      "usd_price": 6360.91
    },
    {
      "start_date": "17/10/2023",
      "usd_price": 6574.8
    },
    {
      "start_date": "16/02/2024",
      "usd_price": 6657.12
    },
    {
      "start_date": "22/02/2024",
      "usd_price": 6703.2
```

```
    },
    {
      "start_date": "15/03/2024",
      "usd_price": 6633.66
    },
    {
      "start_date": "02/08/2024",
      "usd_price": 5774.5
    },
    {
      "start_date": "08/04/2025",
      "usd_price": 5579.75
    }
  ]
}
```


8.2 Normalização para Estrutura Tabular

O campo `historic_data` foi expandido para gerar um registro individual por entrada, resultando em uma estrutura tabular com os campos `product_id`, `product_name`, `category`, `start_date` e `usd_price`.

- **Explosão dos dados:** Iteração sobre cada produto e cada entrada do campo `historic_data`, gerando registros independentes por data.
- **Ajustes de tipo:** Conversão de `product_id` para `int64` e de `start_date` para `datetime64` via `format='mixed'` com `dayfirst=True`.
- **Ordenação:** Registros ordenados por `product_id` e `start_date`, garantindo consistência na identificação do custo vigente em cada data de venda.

> Insight: A ordenação por produto e data é um requisito técnico para garantir que o filtro `start_date <= data_venda` identifique corretamente o último custo vigente antes de cada transação.

In [8]:
```
# ── 8.2 Normalização para Estrutura Tabular ──────────
registros = []

for produto in raw:
    product_id = produto['product_id']
    product_name = produto['product_name']
    category = produto['category']

    for entrada in produto['historic_data']:
        registros.append({
            'product_id': product_id,
            'product_name': product_name,
            'category': category,
            'start_date': entrada['start_date'],
            'usd_price': entrada['usd_price']
        })

df_custos = pd.DataFrame(registros)
```

```python
# Garantir tipos corretos
df_custos['product_id'] = df_custos['product_id'].astype(int)
df_custos['start_date'] = pd.to_datetime(
    df_custos['start_date'],
    format='mixed',
    dayfirst=True
)

# Ordenar por produto e data
df_custos = df_custos.sort_values(['product_id', 'start_date'])

print(f"Total de entradas no CSV: {len(df_custos)}")
print("\nSchema:")
print(df_custos.dtypes)
print("Valores nulos totais:", df_custos.isnull().sum().sum())
display(df_custos.head(5))

# Salvar CSV
df_custos.to_csv('data/custos_importacao_normalizado.csv', index=False)
```

```
Total de entradas no CSV: 1260

Schema:
product_id               int64
product_name               str
category                   str
start_date      datetime64[us]
usd_price              float64
dtype: object
Valores nulos totais: 0
```

	product_id	product_name	category	start_date	usd_price
0	1	Transponder AIS Maré Magnum	eletrônicos	2016-08-10	10583.63
1	1	Transponder AIS Maré Magnum	eletrônicos	2018-06-15	8778.36
2	1	Transponder AIS Maré Magnum	eletrônicos	2018-09-25	8023.87
3	1	Transponder AIS Maré Magnum	eletrônicos	2019-03-19	8772.78
4	1	Transponder AIS Maré Magnum	eletrônicos	2020-01-17	7918.18

8.3 Validação do CSV Gerado

O CSV normalizado foi validado em três dimensões para garantir a integridade da transformação.

- **Volume:** 1.260 registros gerados, refletindo a expansão dos históricos de preço dos 150 produtos da base.
- **Tipos:** Schema consistente com `product_id` em `int64`, `start_date` em `datetime64` e `usd_price` em `float64`.

- **Nulos:** Ausência de valores nulos em todas as colunas, confirmando o correto preenchimento dos campos obrigatórios.

> Insight: A média de aproximadamente 8,4 entradas de preço por produto indica variação frequente nos custos de importação ao longo do tempo, reforçando a necessidade de utilizar o custo vigente na data de cada venda, e não um valor fixo por produto.

8.4 Interpretação — Normalização JSON para Tabular

A estratégia de normalização foi definida com base na estrutura original dos dados e nos requisitos das análises subsequentes.

- **Explosão por iteração:** A abordagem iterativa foi preferida ao `pd.json_normalize()` por oferecer maior controle sobre a estrutura gerada e permitir validações mais granulares por registro.
- **`format='mixed'` com `dayfirst=True`:** Necessário devido à presença de múltiplos formatos de data na base original, garantindo parsing correto sem perda de registros.
- **Ordenação temporal:** Pré-requisito para a correta identificação do custo vigente por meio do filtro `start_date <= data_venda` nas análises de margem.

> Insight: A normalização transforma um arquivo de armazenamento em uma base analítica. Essa separação é fundamental em arquiteturas de dados modernas, onde as camadas de ingestão e análise devem ser tratadas de forma independente.

---

9. Análise de Margem e Identificação de Prejuízo por Produto

O sistema de vendas registra receitas em BRL, enquanto o catálogo de fornecedores armazena custos em USD. Como o câmbio varia diariamente, o custo real de cada venda só pode ser calculado com precisão utilizando a taxa de câmbio vigente na data da transação.

> Insight: 62,8% das transações do período foram realizadas abaixo do custo de importação, com R$ 39,9 milhões de prejuízo concentrados no produto 72, indicando ausência de uma política de precificação dinâmica alinhada à variação cambial.

9.1 Obtenção da Taxa de Câmbio (USD → BRL)

A taxa de câmbio foi obtida por meio da API pública do Banco Central do Brasil (PTAX — endpoint `CotacaoDolarDia`) para cada data de venda.

- **Fonte:** API oficial do BCB, cotação de venda (`cotacaoVenda`).
- **Tratamento de falhas:** Uso de `try/except` para lidar com instabilidades na requisição e garantir continuidade do processamento.
- **Retorno:** Seleção do último valor disponível no dia (`dados[-1]['cotacaoVenda']`), representando a cotação de fechamento.

> Insight: O uso da cotação oficial do BCB garante rastreabilidade e auditabilidade no cálculo de margem, eliminando dependência de fontes externas não confiáveis.

In [9]:
```
# ── 9.1 Obtenção da Taxa de Câmbio (USD → BRL) ──────────
def get_cambio_bcb(data):
    """Busca a taxa de câmbio USD/BRL do BCB para uma data específica."""
    data_str = data.strftime('%m-%d-%Y')
    url = (f"https://olinda.bcb.gov.br/olinda/servico/PTAX/versao/v1/odata/"
           f"CotacaoDolarDia(dataCotacao=@dataCotacao)"
           f"?@dataCotacao='{data_str}'&$format=json&$select=cotacaoVenda")
    try:
        resp = requests.get(url, timeout=10)
        dados = resp.json().get('value', [])
        if dados:
            return dados[-1]['cotacaoVenda']
        return None
    except:
        return None

# Teste com uma data
teste = get_cambio_bcb(pd.Timestamp('2024-01-15'))
print(f"Câmbio teste (15/01/2024): R$ {teste}")
```

```
Câmbio teste (15/01/2024): R$ 4.8765
```

9.2 Construção do Cache de Taxas de Câmbio

Para evitar requisições redundantes à API, as taxas foram consultadas uma única vez por data e armazenadas em `cache_cambio`.

- **Escopo:** 726 datas únicas identificadas na base de vendas.
- **Cobertura:** 100% das datas obtiveram taxa de câmbio após o tratamento de exceções.
- **Fallback:** Para datas sem cotação disponível (fins de semana e feriados bancários), foi aplicada busca retroativa de até 4 dias úteis anteriores.

> Insight: A estratégia de cache reduz o número de requisições à API de 9.895 para 726 (redução de 92,7%), aumentando significativamente a eficiência do processamento. O fallback retroativo garante cobertura total

> sem comprometer a precisão, pois o BCB não divulga PTAX em dias não úteis e o último valor disponível representa a referência de mercado mais próxima.

In [10]:
```
# ── 9.2 Construção do Cache de Taxas de Câmbio ──────────
datas_unicas = df_vendas['sale_date'].dt.date.unique()
print(f"Total de datas únicas para buscar: {len(datas_unicas)}")

cache_cambio = {}
datas_sem_cambio = []

for i, data in enumerate(sorted(datas_unicas)):
    ts = pd.Timestamp(data)
    taxa = get_cambio_bcb(ts)

    if taxa:
        cache_cambio[data] = taxa
    else:
        # Fins de semana e feriados não têm cotação — busca dia anterior
        for dias_atras in range(1, 5):
            ts_anterior = ts - pd.Timedelta(days=dias_atras)
            taxa = get_cambio_bcb(ts_anterior)
            if taxa:
                cache_cambio[data] = taxa
                break
        if data not in cache_cambio:
            datas_sem_cambio.append(data)

    if (i + 1) % 50 == 0:
        print(f"Progresso: {i+1}/{len(datas_unicas)} datas processadas...")

print(f"\nDatas com câmbio encontrado: {len(cache_cambio)}")
print(f"Datas sem câmbio: {len(datas_sem_cambio)}")
```

```
Total de datas únicas para buscar: 726
Progresso: 50/726 datas processadas...
Progresso: 100/726 datas processadas...
Progresso: 150/726 datas processadas...
Progresso: 200/726 datas processadas...
Progresso: 250/726 datas processadas...
Progresso: 300/726 datas processadas...
Progresso: 350/726 datas processadas...
Progresso: 400/726 datas processadas...
Progresso: 450/726 datas processadas...
Progresso: 500/726 datas processadas...
Progresso: 550/726 datas processadas...
Progresso: 600/726 datas processadas...
Progresso: 650/726 datas processadas...
Progresso: 700/726 datas processadas...

Datas com câmbio encontrado: 726
Datas sem câmbio: 0
```

9.3 Cálculo de Custo em BRL e Identificação de Prejuízo

O custo de cada transação em BRL foi calculado em três etapas sequenciais, integrando câmbio, custo histórico e volume vendido.

- **Mapeamento do câmbio:** A taxa de câmbio da data da venda foi associada a cada transação via `map()` sobre o dicionário `cache_cambio`.
- **Custo vigente em USD:** A função `get_custo_vigente()` recupera o último preço registrado em `custos_importacao` com `start_date <= data_venda`, garantindo que o custo utilizado seja o vigente no momento da transação.
- **Custo total em BRL:** Calculado como `custo_usd × taxa_cambio × qtd`, refletindo o custo total da quantidade vendida convertido para moeda local.

> Insight: A flag `tem_prejuizo` criada nesta etapa é o elemento central da análise de margem, pois permite segregar transações lucrativas de deficitárias e viabiliza todas as análises agregadas subsequentes.

In [11]:
```
# ── 9.3 Cálculo de Custo em BRL e Identificação de Prejuízo ──────────
# Mapear câmbio para cada venda
df_vendas['taxa_cambio'] = df_vendas['sale_date'].dt.date.map(cache_cambio)

# Encontrar custo vigente na data da venda (último preço antes ou igual à da
def get_custo_vigente(product_id, data_venda):
    custos_produto = df_custos[
        (df_custos['product_id'] == product_id) &
        (df_custos['start_date'].dt.date <= data_venda)
    ].sort_values('start_date', ascending=False)

    if len(custos_produto) > 0:
        return custos_produto.iloc[0]['usd_price']
    return None

# Aplicar (pode demorar alguns minutos)
df_vendas['custo_usd'] = df_vendas.apply(
    lambda row: get_custo_vigente(row['id_product'], row['sale_date'].date()
    axis=1
)

# Calcular custo em BRL
df_vendas['custo_brl'] = df_vendas['custo_usd'] * df_vendas['taxa_cambio'] *

# Identificar prejuízo (venda abaixo do custo)
df_vendas['prejuizo'] = df_vendas['custo_brl'] - df_vendas['total']
df_vendas['tem_prejuizo'] = df_vendas['prejuizo'] > 0

print(f"Transações com prejuízo: {df_vendas['tem_prejuizo'].sum()}")
print(f"Transações sem prejuízo: {(~df_vendas['tem_prejuizo']).sum()}")
print(f"\nNulos em custo_usd: {df_vendas['custo_usd'].isnull().sum()}")

display(df_vendas[df_vendas['tem_prejuizo']].head(3))
```

```
Transações com prejuízo: 6213
Transações sem prejuízo: 3682

Nulos em custo_usd: 0
```

	id	id_client	id_product	qtd	total	sale_date	taxa_cambio	custo_usd	custo_l
0	0	42	105	11	3405.00	2023-10-09	5.17	62.64	3559.!
1	1	3	136	9	16873.90	2024-09-15	5.57	390.62	19587.
2	2	25	139	7	9475.30	2024-08-13	5.49	246.89	9483.(

9.4 Análise de Prejuízo por Produto

Os dados foram agregados por `id_product` para identificar os produtos com maior impacto financeiro negativo.

- **`receita_total`**: Soma de `total` considerando todas as transações, independentemente de lucro ou prejuízo.
- **`prejuizo_total`**: Soma de `prejuizo` considerando apenas transações com `tem_prejuizo = True`.
- **`percentual_perda`**: Razão entre `prejuizo_total` e `receita_total`, indicando a proporção da receita comprometida por prejuízo operacional.

> Insight: O produto `id_product = 72` lidera em ambas as dimensões — maior prejuízo absoluto (R$ 39,9 milhões) e maior percentual de perda (63,37%) — evidenciando um problema estrutural de precificação, e não ocorrências pontuais.

In [12]:
```python
# ── 9.4 Análise de Prejuízo por Produto ──────────
df_prejuizo_produto = df_vendas.groupby('id_product').agg(
    receita_total    = ('total', 'sum'),
    prejuizo_total   = ('prejuizo', lambda x: x[x > 0].sum()),
    qtd_transacoes   = ('id', 'count')
).reset_index()

df_prejuizo_produto['percentual_perda'] = (
    df_prejuizo_produto['prejuizo_total'] /
    df_prejuizo_produto['receita_total']
)

# Apenas produtos com prejuízo
df_com_prejuizo = df_prejuizo_produto[
    df_prejuizo_produto['prejuizo_total'] > 0
].sort_values('prejuizo_total', ascending=False)

print(f"Produtos com prejuízo: {len(df_com_prejuizo)}")
print(f"\nTop 5 — maior prejuízo absoluto:")
```

```python
display(df_com_prejuizo.head(5)[['id_product','receita_total',
                                 'prejuizo_total','percentual_perda']])

print(f"\nProduto com MAIOR prejuízo absoluto:")
top_abs = df_com_prejuizo.iloc[0]
print(f"  id_product: {top_abs['id_product']}")
print(f"  Prejuízo total: R$ {top_abs['prejuizo_total']:,.2f}")

print(f"\nProduto com MAIOR % de perda:")
top_pct = df_com_prejuizo.sort_values('percentual_perda', ascending=False).i
print(f"  id_product: {top_pct['id_product']}")
print(f"  Percentual de perda: {top_pct['percentual_perda']:.2%}")
```

```
Produtos com prejuízo: 150

Top 5 — maior prejuízo absoluto:
```

	id_product	receita_total	prejuizo_total	percentual_perda
71	72	63057815.65	39959694.85	0.63
82	83	44377440.00	18382383.33	0.41
73	74	59764356.15	7004602.39	0.12
70	71	81567066.65	6120379.87	0.08
54	55	61224375.00	5597894.25	0.09

```
Produto com MAIOR prejuízo absoluto:
  id_product: 72.0
  Prejuízo total: R$ 39,959,694.85

Produto com MAIOR % de perda:
  id_product: 72.0
  Percentual de perda: 63.37%
```

9.5 Validação da Análise via SQL (DuckDB)

Os resultados obtidos em Python foram reproduzidos via SQL com DuckDB como forma de validação independente.

- **Método:** Query SQL com `WITH custo_vigente AS (...)`, reproduzindo as métricas `receita_total`, `prejuizo_total` e `percentual_perda` com os mesmos critérios de cálculo, filtro e ordenação.
- **Escopo:** Top 10 produtos por `prejuizo_total`, considerando apenas registros com `prejuizo_total > 0`.
- **Resultado:** Total consistência entre os valores obtidos em Python e SQL em todas as métricas analisadas.

> Insight: A consistência entre Python e SQL valida a implementação da lógica de negócio e aumenta a confiabilidade dos resultados para suporte à tomada de decisão.

In [13]:
```python
# ── 9.5 Validação da Análise via SQL (DuckDB) ──────────────────────────
conn.register('df_vendas', df_vendas)
conn.register('df_custos', df_custos)

query_q4 = """
WITH custo_vigente AS (
    SELECT
        v.id        AS venda_id,
        v.id_product,
        v.total,
        v.taxa_cambio,
        v.qtd,
        v.custo_usd,
        v.custo_brl,
        v.prejuizo,
        v.tem_prejuizo
    FROM df_vendas v
)
SELECT
    id_product,
    ROUND(SUM(total), 2)                                AS receita_total,
    ROUND(SUM(CASE WHEN tem_prejuizo THEN prejuizo
                   ELSE 0 END), 2)                      AS prejuizo_total,
    ROUND(SUM(CASE WHEN tem_prejuizo THEN prejuizo
                   ELSE 0 END) / SUM(total), 4)         AS percentual_perda
FROM custo_vigente
GROUP BY id_product
HAVING prejuizo_total > 0
ORDER BY prejuizo_total DESC
LIMIT 10
"""

resultado_q4 = conn.execute(query_q4).df()
display(resultado_q4)
```

	id_product	receita_total	prejuizo_total	percentual_perda
0	72	63057815.65	39959694.85	0.63
1	83	44377440.00	18382383.33	0.41
2	74	59764356.15	7004602.39	0.12
3	71	81567066.65	6120379.87	0.07
4	55	61224375.00	5597894.25	0.09
5	91	66829268.70	5432887.33	0.08
6	63	68817185.90	4614028.37	0.07
7	78	58043733.35	4500947.90	0.08
8	60	53418532.00	3035102.62	0.06
9	96	27227868.90	2991965.91	0.11

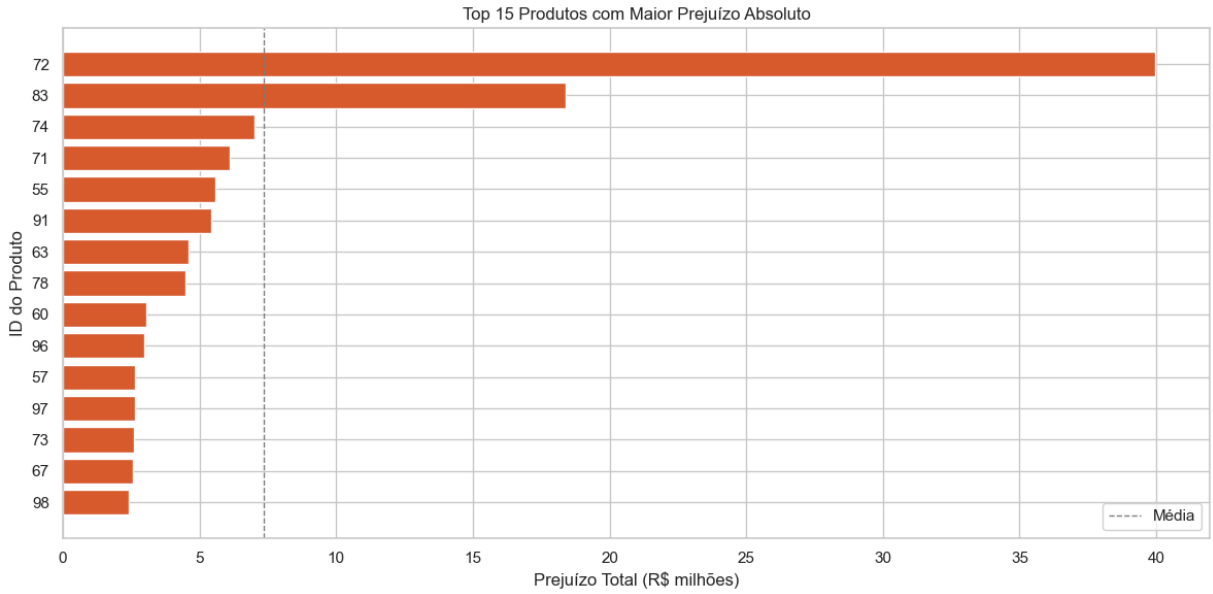
9.6 Interpretação — Análise de Prejuízo com Câmbio Real

A metodologia aplicada garante rastreabilidade e consistência no cálculo de margem.

- **Câmbio:** Cotação oficial BCB/PTAX de venda na data da transação, com fallback retroativo de até 4 dias úteis para datas sem cotação disponível.
- **Custo vigente:** Último preço registrado em `custos_importacao` com `start_date <= data_venda`, garantindo que o custo utilizado seja o vigente no momento da transação.
- **Suposições adotadas:** O custo total foi calculado como `custo_usd × taxa_cambio × qtd`. Impostos e frete foram desconsiderados conforme premissa do projeto.

> Insight: O padrão de prejuízo observado em 62,8% das transações não decorre de eventos pontuais, mas de uma defasagem sistemática entre os preços de venda praticados e a variação cambial acumulada, indicando ausência de uma política de precificação dinâmica alinhada ao câmbio.

In [14]:
```
# ── 9.6 Visual 1 — Prejuízo total por produto (Top 15) ──────
fig, ax = plt.subplots(figsize=(12, 6))
top15 = df_com_prejuizo.head(15).sort_values('prejuizo_total')
ax.barh(
    top15['id_product'].astype(str),
    top15['prejuizo_total'] / 1e6,
    color='#D85A30'
)
ax.set_xlabel('Prejuízo Total (R$ milhões)')
ax.set_ylabel('ID do Produto')
ax.set_title('Top 15 Produtos com Maior Prejuízo Absoluto')
ax.axvline(
    top15['prejuizo_total'].mean() / 1e6,
    color='gray', linestyle='--', linewidth=1,
    label='Média'
)
ax.legend()
plt.tight_layout()
plt.savefig('outputs/visual_prejuizo_produtos.png', dpi=150, bbox_inches='ti
plt.show()
```



9.7 Conclusão — Análise de Prejuízo por Produto

A análise evidencia um problema estrutural de precificação com impacto financeiro significativo e mensurável.

- **Escala:** 6.213 das 9.895 transações (62,8%) foram realizadas abaixo do custo de importação no período analisado.
- **Concentração:** O produto `id_product = 72` acumula R$ 39,9 milhões em prejuízo absoluto, representando 63,37% de sua receita total.
- **Rastreabilidade:** O uso de câmbio oficial BCB/PTAX, custo vigente por data e validação cruzada entre Python e SQL garante a confiabilidade dos resultados.

> Insight: A correção do problema não depende de novos dados, mas da implementação de uma política de precificação dinâmica que ajuste os preços de venda conforme a variação cambial, utilizando a metodologia desenvolvida nesta análise como base de cálculo.

---

10. Análise de Clientes com Foco em Fidelização e Rentabilidade

Identificar clientes de alto valor vai além de ranquear por faturamento total. Um cliente com alto faturamento concentrado em poucas transações pode ser menos estratégico do que um cliente com alta frequência e diversidade de categorias.

> Insight: Os 10 clientes elite identificados apresentam ticket médio entre R $290.063 e R$ 336.859, todos com diversidade de 3 categorias, indicando que alto valor e comportamento multidisciplinar são atributos correlacionados nesse segmento.

10.1 Metodologia e Critérios de Segmentação

As métricas foram calculadas por cliente para permitir a segmentação por valor e comportamento de compra.

- **`ticket_medio`:** Razão entre `faturamento_total` e `frequencia`, reduzindo o impacto de transações isoladas de alto valor.
- **`diversidade_categorias`:** Contagem de categorias distintas compradas, calculada após a padronização aplicada na seção 7.
- **Filtro de elite:** Seleção de clientes com `diversidade_categorias >= 3`, garantindo que o ranking reflita comportamento multidisciplinar consistente.

> Insight: O critério de diversidade mínima de 3 categorias diferencia clientes estratégicos de compradores recorrentes especializados, tornando o ranking mais relevante para decisões de retenção e expansão de carteira.

In [15]:
```
# —— 10.1 Metodologia e Critérios de Segmentação (Análise de Clientes com Fo
conn.register('df_vendas', df_vendas)
conn.register('df_produtos', df_produtos)

query_q5 = """
WITH vendas_com_categoria AS (
    SELECT
        v.id_client,
        v.id,
        v.total,
        p.actual_category
    FROM df_vendas v
    LEFT JOIN df_produtos p
        ON v.id_product = p.code
),
metricas_cliente AS (
    SELECT
```

```
        id_client,
        SUM(total)                          AS faturamento_total,
        COUNT(DISTINCT id)                  AS frequencia,
        ROUND(SUM(total) / COUNT(DISTINCT id), 2) AS ticket_medio,
        COUNT(DISTINCT actual_category)     AS diversidade_categorias
    FROM vendas_com_categoria
    GROUP BY id_client
),
clientes_elite AS (
    SELECT *
    FROM metricas_cliente
    WHERE diversidade_categorias >= 3
    ORDER BY ticket_medio DESC, id_client ASC
    LIMIT 10
)
SELECT * FROM clientes_elite
"""

df_clientes_elite = conn.execute(query_q5).df()
display(df_clientes_elite)
```

	id_client	faturamento_total	frequencia	ticket_medio	diversidade_categorias
0	47	64003343.75	190	336859.70	3
1	42	72187369.50	222	325168.33	3
2	9	66788855.35	218	306370.90	3
3	22	59581398.75	198	300916.16	3
4	2	65652931.35	220	298422.42	3
5	28	60826837.25	204	298170.77	3
6	46	59126834.35	199	297119.77	3
7	38	57093331.15	195	292786.31	3
8	36	62791038.15	215	292051.34	3
9	5	58592802.70	202	290063.38	3

10.2 Top 10 Clientes Elite

Os 10 clientes identificados apresentam ticket médio mínimo de R $290.063,38 e máximo de R$ 336.859,70, todos com exatamente 3 categorias distintas de compra.

- **Ticket médio:** Entre 10% e 27% acima da média geral da base (R$ 263.797,83).
- **Diversidade:** Todos os 10 clientes apresentam exatamente 3 categorias distintas, sem ocorrência de clientes com 4 ou mais categorias no ranking.
- **Frequência:** Entre 190 e 222 transações por cliente no período analisado.

> Insight: O fato de todos os 10 clientes apresentarem exatamente 3 categorias indica que o comportamento de compra multicanal é consistente nesse perfil, e não resultado de transações pontuais.

In [16]:
```
# ── 10.2 Top 10 Clientes Elite ──────────
top10_ids = df_clientes_elite['id_client'].tolist()

query_q5_categoria = """
WITH vendas_top10 AS (
    SELECT
        v.id_client,
        p.actual_category,
        SUM(v.qtd) AS total_itens
    FROM df_vendas v
    LEFT JOIN df_produtos p
        ON v.id_product = p.code
    WHERE v.id_client IN (
        SELECT id_client FROM df_clientes_elite
    )
    GROUP BY v.id_client, p.actual_category
)
SELECT
    actual_category,
    SUM(total_itens) AS total_itens
FROM vendas_top10
GROUP BY actual_category
ORDER BY total_itens DESC
"""

conn.register('df_clientes_elite', df_clientes_elite)
df_categoria_top10 = conn.execute(query_q5_categoria).df()
display(df_categoria_top10)
```

	actual_category	total_itens
0	propulsão	6030.00
1	ancoragem	5632.00
2	eletrônicos	5214.00

In [17]:
```
#── 10.2 – Enriquecimento com dados do CRM ─────────────────
import json

with open('data/clientes_crm.json', 'r', encoding='utf-8') as f:
    crm = json.load(f)

df_crm = pd.DataFrame(crm).rename(columns={'code': 'id_client'})

# Padronizar localização
def padronizar_local(loc):
    loc = str(loc).strip()
    loc = loc.replace(' / ', ', ').replace(' - ', ', ').replace('/', ', ')
    loc = loc.replace(' , ', ', ').replace(',', ', ')
```

```
    return loc

df_crm['location'] = df_crm['location'].apply(padronizar_local)

# Enriquecer Top 10 com dados do CRM
df_elite_crm = df_clientes_elite.merge(
    df_crm[['id_client', 'full_name', 'location']],
    on='id_client',
    how='left'
)

# Exibir tabela enriquecida
display(df_elite_crm[['id_client', 'full_name', 'location',
                        'ticket_medio', 'frequencia',
                        'diversidade_categorias']])
```

	id_client	full_name	location	ticket_medio	frequencia	diversidade_categoria
0	47	Gabriela Barros Lacerda	RJ, Niterói	336859.70	190	
1	42	Márcia Figueiredo	Vila do Conde (Barcarena), PA	325168.33	222	
2	9	Lucas Guedes Cunha Lopes	PB, João Pessoa	306370.90	218	
3	22	Daniela Borges Vieira Farias Mendonça	SE, Aracaju	300916.16	198	
4	2	Fernanda Azevedo Soares Nunes Vieira	PE, Recife	298422.42	220	
5	28	Bianca Rodrigues	PR, Antonina	298170.77	204	
6	46	Ana Silva Costa Farias Coelho	Macapá, AP	297119.77	199	
7	38	Mateus Antunes	TO, Fortaleza do Tabocão	292786.31	195	
8	36	Francisca Ribeiro Pinheiro	Belém, PA	292051.34	215	
9	5	Pedro Freitas	PA, Santarém Novo	290063.38	202	

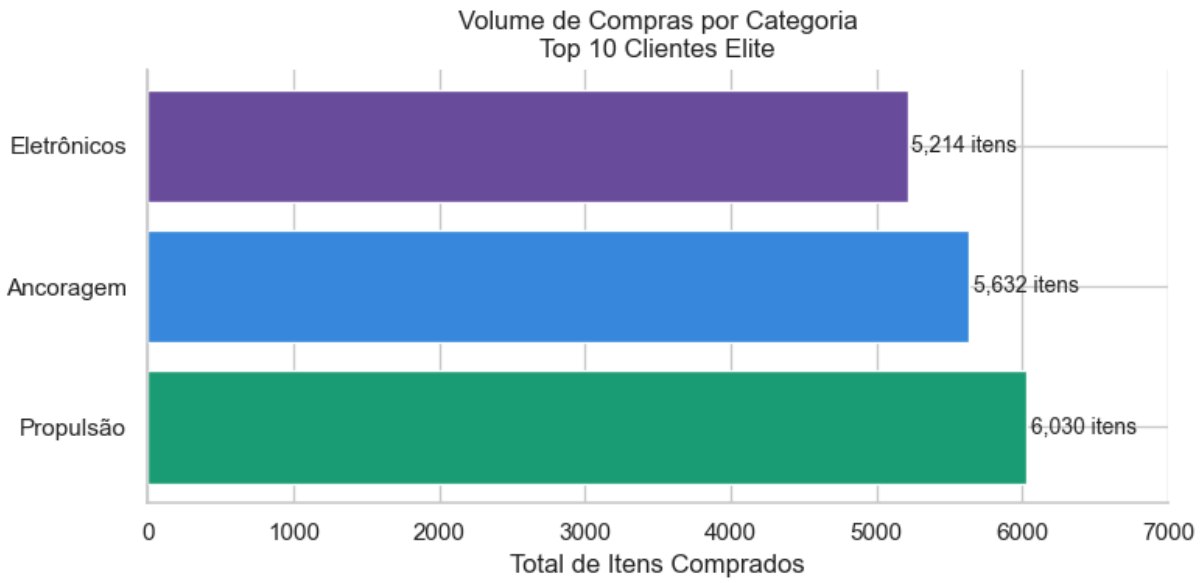
10.3 Categoria Mais Vendida entre os Clientes Elite

A análise considera exclusivamente as transações dos 10 clientes elite, agregando o total de itens por categoria via `SUM(qtd)`.

- **Propulsão:** 6.030 itens — categoria com maior volume.
- **Ancoragem:** 5.632 itens.
- **Eletrônicos:** 5.214 itens.

> Insight: A distribuição equilibrada entre as três categorias confirma o perfil multidisciplinar desse grupo. A liderança de propulsão sugere que essa categoria atua como porta de entrada para clientes de alto valor, sendo estratégica em iniciativas de aquisição e cross-sell.

In [18]:
```
# ── 10.3 Visual 2 — Categoria Mais Vendida entre os Clientes Elite (Volume
fig, ax = plt.subplots(figsize=(8, 4))

categorias = ['Propulsão', 'Ancoragem', 'Eletrônicos']
valores = [6030, 5632, 5214]
cores = ['#1D9E75', '#378ADD', '#6C4E9E']

bars = ax.barh(categorias, valores, color=cores)

for bar, val in zip(bars, valores):
    ax.text(
        bar.get_width() + 30,
        bar.get_y() + bar.get_height() / 2,
        f'{val:,} itens',
        va='center',
        fontsize=10
    )

ax.set_xlabel('Total de Itens Comprados')
ax.set_title('Volume de Compras por Categoria\nTop 10 Clientes Elite')
ax.set_xlim(0, 7000)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.savefig('outputs/visual_clientes_elite.png', dpi=150, bbox_inches='tight
plt.show()
```



10.4 Interpretação — Análise de Clientes Fiéis

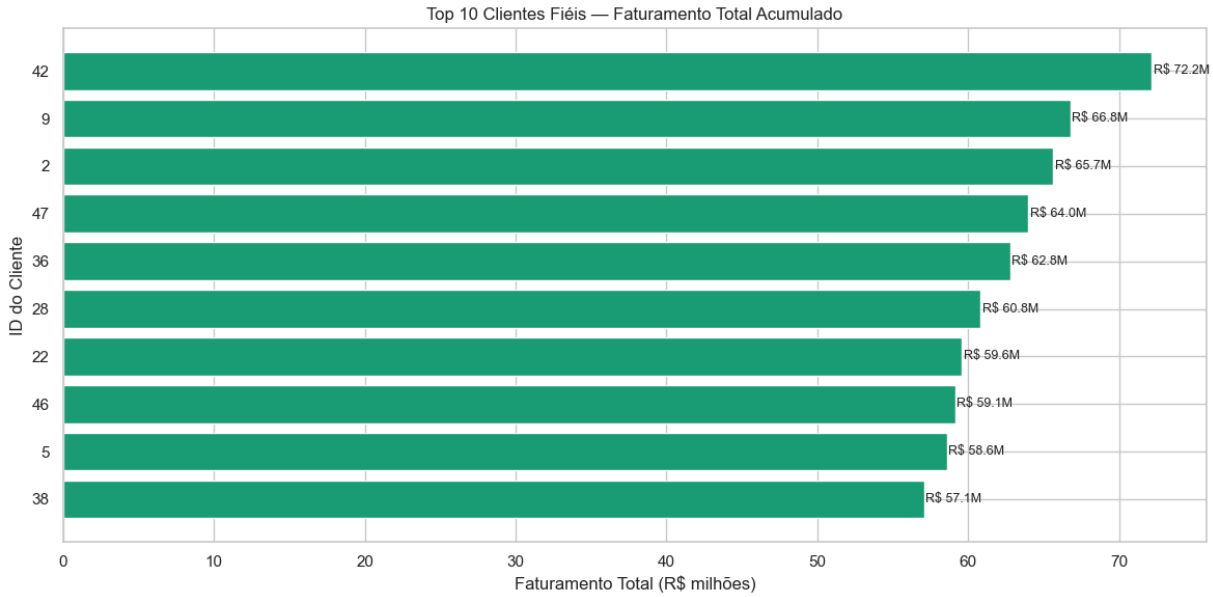
A metodologia de segmentação combina valor e comportamento para identificar o perfil de cliente mais estratégico para a LH Nautical.

- **Ticket médio como métrica principal:** Mais resistente a distorções que o faturamento total, pois normaliza o valor por frequência e reduz o impacto de transações isoladas de alto valor.
- **Filtro de diversidade:** A exigência de 3 ou mais categorias garante que o ranking reflita clientes com comportamento de compra amplo, evitando a dominância de compradores especializados de alto valor.
- **Desempate por `id_client`:** Critério determinístico que garante reprodutibilidade do ranking, independentemente da ordem de processamento.

> Insight: A combinação de alto ticket médio, alta frequência e diversidade de categorias define o cliente ideal da LH Nautical. Estratégias de retenção direcionadas a esse perfil tendem a gerar maior impacto na receita recorrente do que ações voltadas a clientes com faturamento elevado, porém concentrado.

In [19]:
```
# ── 10.4 Visual 3 – Faturamento acumulado dos Top 10 clientes –
fig, ax = plt.subplots(figsize=(12, 6))

df_plot = df_clientes_elite.sort_values('faturamento_total', ascending=True)

bars = ax.barh(
    df_plot['id_client'].astype(str),
    df_plot['faturamento_total'] / 1e6,
    color='#1D9E75'
)

# Adicionar valores nas barras
```

```
for bar, val in zip(bars, df_plot['faturamento_total'] / 1e6):
    ax.text(
        bar.get_width() + 0.1,
        bar.get_y() + bar.get_height() / 2,
        f'R$ {val:.1f}M',
        va='center', fontsize=9
    )

ax.set_xlabel('Faturamento Total (R$ milhões)')
ax.set_ylabel('ID do Cliente')
ax.set_title('Top 10 Clientes Fiéis — Faturamento Total Acumulado')
plt.tight_layout()
plt.savefig('outputs/visual_vendas_dia_semana.png', dpi=150, bbox_inches='ti
plt.show()
```



10.5 Conclusão — Análise de Clientes Fiéis

Os 10 clientes elite representam o perfil mais estratégico da base por combinarem simultaneamente três atributos de alto valor.

- **Recorrência:** Entre 190 e 222 transações por cliente no período, gerando receita previsível e consistente.
- **Valor:** Ticket médio entre 10% e 27% acima da média geral da base.
- **Diversidade:** Consumo distribuído pelas três categorias da loja, reduzindo o risco de churn associado à dependência de uma única linha de produtos.

> Insight: A categoria propulsão lidera o volume de itens entre os clientes elite e atua como principal porta de entrada para esse perfil. Estratégias de aquisição focadas em compradores dessa categoria, com incentivo à expansão para ancoragem e eletrônicos, representam o caminho mais direto para ampliar o segmento elite da base.

---

11. Análise Temporal com Calendário Completo

Um erro comum em análises de sazonalidade é calcular médias diretamente sobre a tabela de vendas utilizando `GROUP BY` por dia da semana. Essa abordagem ignora os dias em que a loja operou sem registrar vendas, inflando artificialmente as médias.

> Insight: O domingo apresenta a menor média de vendas (R$ 3.229.614,16), resultado confiável apenas porque a análise considerou todos os dias do período, incluindo dias sem vendas.

11.1 Construção do Calendário e Cálculo da Média por Dia da Semana

A dimensão de datas foi construída via `generate_series` no DuckDB, cobrindo todos os dias entre a data mínima e máxima de `df_vendas`.

- **`generate_series`:** Gera uma linha para cada dia do período analisado, garantindo que nenhum dia seja excluído do cálculo.
- **`LEFT JOIN`:** Mantém todos os dias do calendário, incluindo aqueles sem registro de vendas.
- **`COALESCE(SUM(v.total), 0)`:** Substitui valores nulos por zero nos dias sem vendas, eliminando viés no cálculo das médias.

> Insight: A combinação de `generate_series`, `LEFT JOIN` e `COALESCE` representa o padrão correto para análises temporais com dias ausentes. A ausência de qualquer um desses elementos resulta em médias distorcidas.

In [20]:
```
# ── 11.1 Construção do Calendário e Cálculo da Média por Dia da Semana ──
query_q6 = """
WITH calendario AS (
    SELECT UNNEST(
        generate_series(
            (SELECT MIN(sale_date) FROM df_vendas),
            (SELECT MAX(sale_date) FROM df_vendas),
            INTERVAL '1 day'
        )
    ) AS data
),
vendas_diarias AS (
    SELECT
        c.data,
        COALESCE(SUM(v.total), 0) AS valor_venda
    FROM calendario c
    LEFT JOIN df_vendas v
        ON c.data::DATE = v.sale_date::DATE
    GROUP BY c.data
```

```
),
dia_semana AS (
    SELECT
        data,
        valor_venda,
        CASE EXTRACT(DOW FROM data)
            WHEN 0 THEN 'Domingo'
            WHEN 1 THEN 'Segunda-feira'
            WHEN 2 THEN 'Terça-feira'
            WHEN 3 THEN 'Quarta-feira'
            WHEN 4 THEN 'Quinta-feira'
            WHEN 5 THEN 'Sexta-feira'
            WHEN 6 THEN 'Sábado'
        END AS nome_dia
    FROM vendas_diarias
)
SELECT
    nome_dia,
    ROUND(AVG(valor_venda), 2)  AS media_vendas,
    COUNT(*)                     AS total_dias
FROM dia_semana
GROUP BY nome_dia
ORDER BY AVG(valor_venda) ASC
"""

df_calendario = conn.execute(query_q6).df()
display(df_calendario)
```

	nome_dia	media_vendas	total_dias
0	Domingo	3229614.16	105
1	Segunda-feira	3484500.47	105
2	Terça-feira	3488871.99	105
3	Quarta-feira	3534007.21	104
4	Quinta-feira	3713299.94	104
5	Sábado	3774290.79	104
6	Sexta-feira	3776151.25	104

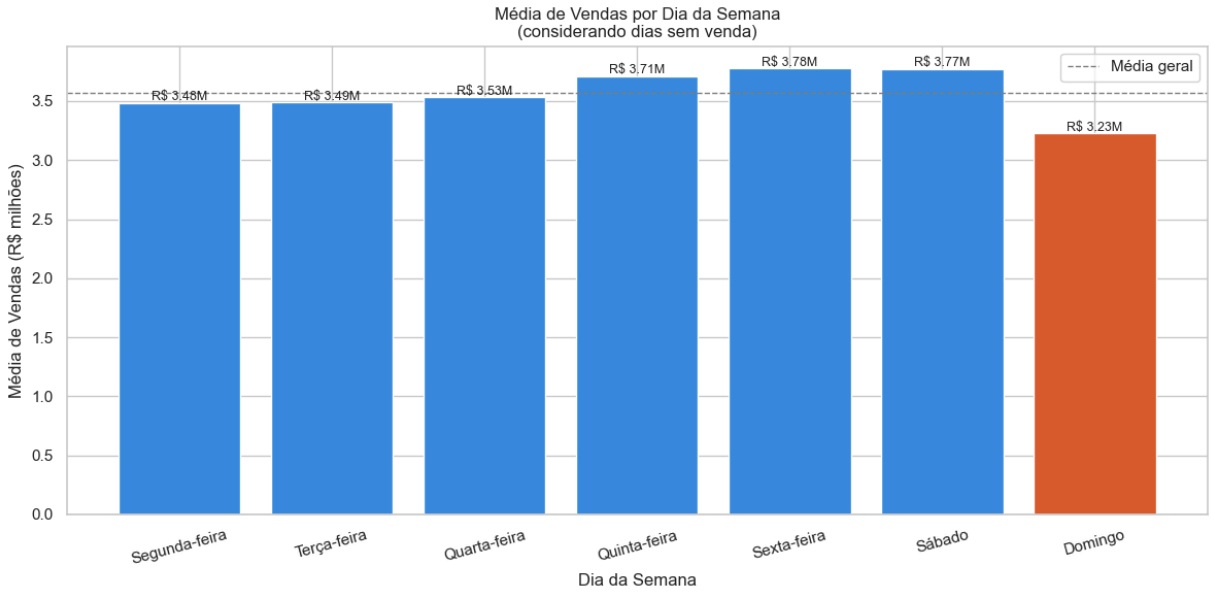
11.2 Interpretação — Dimensão de Calendário

A construção do calendário completo elimina o viés de seleção presente em análises temporais baseadas apenas em dias com vendas.

- **Domingo:** Menor média histórica de vendas — R$ 3.229.614,16.
- **Sexta-feira:** Maior média histórica de vendas — R$ 3.776.151,25.
- **Amplitude:** Diferença de R$ 546.537,09 entre o melhor e o pior dia, relevante para decisões de escalonamento de equipe e reposição de estoque.

> Insight: Sem o calendário completo, dias sem vendas seriam ignorados, inflando as médias de todos os dias da semana e podendo alterar o ranking de performance, o que levaria a decisões operacionais incorretas.

In [21]:
```
# ── 11.2 Visual 4 — Interpretação — Dimensão de Calendário —
ordem_dias = [
    'Segunda-feira', 'Terça-feira', 'Quarta-feira',
    'Quinta-feira', 'Sexta-feira', 'Sábado', 'Domingo'
]

df_plot = df_calendario.set_index('nome_dia').reindex(ordem_dias).reset_inde

fig, ax = plt.subplots(figsize=(12, 6))

cores = ['#D85A30' if d == 'Domingo' else '#378ADD' for d in df_plot['nome_c

bars = ax.bar(
    df_plot['nome_dia'],
    df_plot['media_vendas'] / 1e6,
    color=cores
)

# Valores no topo de cada barra
for bar, val in zip(bars, df_plot['media_vendas'] / 1e6):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.02,
        f'R$ {val:.2f}M',
        ha='center', fontsize=9
    )

ax.set_xlabel('Dia da Semana')
ax.set_ylabel('Média de Vendas (R$ milhões)')
ax.set_title('Média de Vendas por Dia da Semana\n(considerando dias sem venc
ax.axhline(
    df_plot['media_vendas'].mean() / 1e6,
    color='gray', linestyle='--', linewidth=1,
    label='Média geral'
)
ax.legend()
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig('outputs/visual_previsao_demanda.png', dpi=150, bbox_inches='tig
plt.show()
```



11.3 Conclusão — Análise Temporal com Calendário Completo

A construção de uma dimensão de datas completa é um requisito técnico para análises de sazonalidade confiáveis na base da LH Nautical.

- **Resultado:** O domingo é o pior dia de vendas, com média de R$ 3.229.614,16, 14,5% abaixo da sexta-feira.
- **Correção:** A inclusão de dias sem venda corrige o viés inflado das médias, produzindo resultados mais precisos e auditáveis.
- **Padrão:** A abordagem `generate_series` + `LEFT JOIN` + `COALESCE` deve ser adotada como padrão em todos os relatórios temporais da operação.

> Insight: A diferença de R$ 546.537,09 entre o melhor e o pior dia da semana representa uma oportunidade operacional concreta. Ações como escalonamento de equipe, promoções direcionadas e reposição de estoque alinhadas ao calendário de performance podem capturar parte dessa diferença como receita incremental.

---

12. Previsão de Demanda

Decisões de reposição de estoque baseadas em intuição geram dois problemas recorrentes: ruptura de itens de alta rotatividade e excesso de itens de baixa saída. Esta seção documenta a construção de um modelo baseline de previsão de vendas diárias para o produto "Motor de Popa Yamaha Evo Dash 155HP", utilizando média móvel de 7 dias como estimativa para o dia seguinte.

> Insight: O modelo baseline atingiu MAE de 1,64 unidades, estabelecendo um piso de erro que qualquer modelo mais sofisticado deverá superar para

> justificar sua complexidade.

12.1 Preparação dos Dados

Os dados foram filtrados, agregados e estruturados para viabilizar a construção e avaliação do modelo baseline.

- **Produto:** "Motor de Popa Yamaha Evo Dash 155HP" (`id_product = 54`).
- **Agregação:** Consolidação das vendas diárias via `groupby('sale_date')['qtd'].sum()` .
- **Calendário:** Uso de `pd.date_range` com preenchimento (`fill_value=0`) para dias sem venda, garantindo continuidade temporal da série.
- **Divisão:** Treino até 31/12/2023 (356 dias) e teste em janeiro de 2024 (31 dias).

> Insight: A criação do calendário completo para o produto segue o mesmo princípio aplicado na seção 11 — dias sem venda não podem ser ignorados sem comprometer a integridade da janela de 7 dias da média móvel.

In [22]:
```
# ── 12. Previsão de Demanda ──────────────
PRODUTO = 'Motor de Popa Yamaha Evo Dash 155HP'

# Filtrar produto pelo nome
id_produto = df_produtos[df_produtos['name'] == PRODUTO]['code'].values[0]
print(f"ID do produto: {id_produto}")

# Agregar vendas diárias
df_produto = df_vendas[df_vendas['id_product'] == id_produto].copy()
df_diario = df_produto.groupby('sale_date')['qtd'].sum().reset_index()
df_diario.columns = ['data', 'qtd_vendida']

# Criar calendário completo para o produto
datas_completas = pd.date_range(
    start=df_diario['data'].min(),
    end='2024-01-31',
    freq='D'
)
df_diario = (df_diario.set_index('data')
             .reindex(datas_completas, fill_value=0)
             .reset_index()
             .rename(columns={'index': 'data'}))

# Separar treino e teste
treino = df_diario[df_diario['data'] <= '2023-12-31'].copy()
teste  = df_diario[df_diario['data'] >= '2024-01-01'].copy()

print(f"Período de treino: {treino['data'].min().date()} a {treino['data'].m
print(f"Período de teste:  {teste['data'].min().date()} a {teste['data'].max
print(f"Dias de treino: {len(treino)}")
print(f"Dias de teste:  {len(teste)}")
```

```
ID do produto: 54
Período de treino: 2023-01-10 a 2023-12-31
Período de teste:  2024-01-01 a 2024-01-31
Dias de treino: 356
Dias de teste:  31
```

12.2 Modelo Baseline — Média Móvel de 7 Dias

O modelo estima a demanda utilizando a média das vendas dos 7 dias anteriores à data prevista.

- **Janela de 7 dias:** Escolhida para capturar o padrão semanal de vendas sem introduzir ruído de períodos mais longos.
- **Anti-leakage:** Filtro `serie_completa.index < data` garante que apenas dados anteriores à data prevista sejam utilizados no cálculo, evitando o uso de informação futura.
- **Métrica:** MAE (Mean Absolute Error), diretamente interpretável como erro médio em unidades de produto por dia.

> Insight: O uso do operador estritamente menor (`<`) é essencial para evitar data leakage. A substituição por `<=` incluiria a própria data prevista no cálculo, contaminando o modelo com informação futura e invalidando a avaliação de performance.

In [23]:
```
# ── 12.2 Modelo Baseline — Média Móvel de 7 Dias ──────────
serie_completa = df_diario.set_index('data')['qtd_vendida']

previsoes = []

for data in teste['data']:
    # Apenas dados anteriores à data — sem data leakage
    historico = serie_completa[serie_completa.index < data].tail(7)
    previsao = historico.mean() if len(historico) > 0 else 0
    previsoes.append({
        'data':      data,
        'real':      serie_completa.get(data, 0),
        'previsao':  round(previsao, 2)
    })

df_previsoes = pd.DataFrame(previsoes)

# Métrica MAE
mae = (df_previsoes['real'] - df_previsoes['previsao']).abs().mean()

print(f"MAE (Mean Absolute Error): {mae:.4f}")
print(f"\nSoma total prevista (01/01 a 07/01): "
      f"{df_previsoes[df_previsoes['data'] <= '2024-01-07']['previsao'].sum(
print(f"Soma total real    (01/01 a 07/01): "
      f"{df_previsoes[df_previsoes['data'] <= '2024-01-07']['real'].sum():.0

display(df_previsoes.head(10))
```

```
MAE (Mean Absolute Error): 1.6413

Soma total prevista (01/01 a 07/01): 3
Soma total real    (01/01 a 07/01): 10
```

	data	real	previsao
0	2024-01-01	0	0.00
1	2024-01-02	0	0.00
2	2024-01-03	0	0.00
3	2024-01-04	0	0.00
4	2024-01-05	10	0.00
5	2024-01-06	0	1.43
6	2024-01-07	0	1.43
7	2024-01-08	0	1.43
8	2024-01-09	0	1.43
9	2024-01-10	0	1.43

12.3 Interpretação — Previsão de Demanda

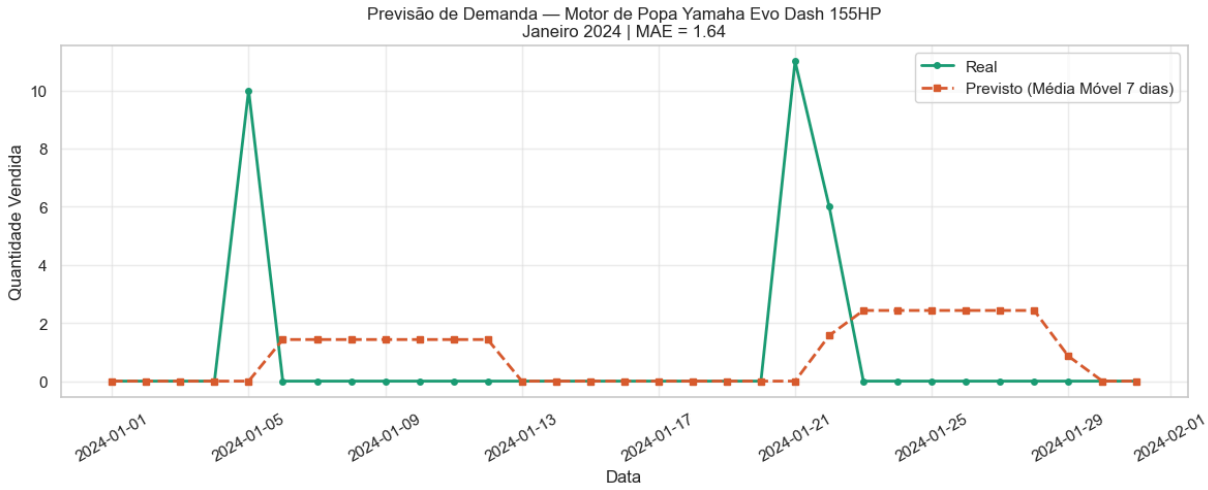
O modelo baseline foi avaliado sobre 31 dias de teste para quantificar sua capacidade preditiva em condições reais.

- **MAE:** 1,64 unidades por dia no período de teste.
- **Primeira semana:** Modelo previu 3 unidades, valor real foi 10, evidenciando a limitação do baseline para capturar picos de demanda.
- **Padrão da série:** Alta frequência de dias com zero vendas e picos esporádicos caracterizam demanda intermitente, padrão para o qual a média móvel simples é inadequada.

> Insight: O MAE de 1,64 não deve ser interpretado isoladamente. Para um produto com demanda intermitente, o erro relevante é a subestimação dos picos — exatamente o que ocorreu na primeira semana, onde o modelo previu 3 unidades contra 10 reais, representando 70% de erro no período mais crítico para decisões de reposição.

In [24]:
```
# —— 12.3 Visual 5 — Previsão de Demanda (Real vs Previsto Janeiro 2024) ——
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    df_previsoes['data'],
    df_previsoes['real'],
    color='#1D9E75',
    linewidth=2,
```

```
    marker='o',
    markersize=4,
    label='Real'
)
ax.plot(
    df_previsoes['data'],
    df_previsoes['previsao'],
    color='#D85A30',
    linewidth=2,
    linestyle='--',
    marker='s',
    markersize=4,
    label='Previsto (Média Móvel 7 dias)'
)

ax.set_xlabel('Data')
ax.set_ylabel('Quantidade Vendida')
ax.set_title(
    'Previsão de Demanda — Motor de Popa Yamaha Evo Dash 155HP\n'
    'Janeiro 2024 | MAE = 1.64'
)
ax.legend()
ax.grid(True, alpha=0.3)
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig('outputs/visual_recomendacao.png', dpi=150, bbox_inches='tight')
plt.show()
```



12.4 Conclusão — Previsão de Demanda

O modelo baseline cumpre seu papel como referência mínima de performance, mas apresenta limitações relevantes para uso operacional.

- **MAE de 1,64:** Define o piso de erro que modelos mais sofisticados devem superar para justificar maior complexidade.
- **Limitação principal:** A média móvel simples não captura sazonalidade, tendências ou picos esporádicos, sendo inadequada para produtos com demanda intermitente.

- **Próximo passo:** Avaliar modelos específicos para séries intermitentes, como Croston ou Prophet com sazonalidade semanal, utilizando o MAE de 1,64 como benchmark.

> Insight: Para produtos com demanda irregular, a principal limitação não está na acurácia média, mas na incapacidade de antecipar picos. Um modelo que subestima sistematicamente a demanda em dias de alta gera maior impacto operacional do que um modelo com MAE mais elevado, porém com erros distribuídos de forma mais equilibrada.

---

13. Sistema de Recomendação por Similaridade de Comportamento de Compra

Clientes que compram determinados produtos tendem a adquirir outros itens de forma recorrente. Esta seção documenta a construção de um sistema de recomendação item-based utilizando similaridade de cosseno sobre uma matriz binária de interações usuário-produto.

> **Insight principal:** O produto mais similar ao "GPS Garmin Vortex Maré Drift" é o "Motor de Popa Volvo Magnum 276HP" (similaridade 0,871), indicando que clientes que investem em navegação de precisão tendem a equipar embarcações com motores de alta potência, sugerindo um perfil de uso voltado a navegação de longo curso.

13.1 Construção da Matriz Usuário x Produto

A matriz de interação foi construída com valores binários para capturar padrões de co-ocorrência de compras, independentemente do volume adquirido.

- **Binarização:** Valor 1 se o cliente comprou o produto ao menos uma vez e 0 caso contrário. A quantidade foi ignorada deliberadamente.
- **Dimensão:** 49 clientes × 150 produtos, resultando em uma matriz esparsa de interações.
- **Transposição:** Aplicação de `matriz.T` para posicionar produtos nas linhas e permitir o cálculo da similaridade produto a produto.

> Insight: A binarização equaliza o peso das interações, evitando que produtos de alta rotatividade dominem a similaridade por volume e não por comportamento de compra.

In [25]:
```
# ── 13.1 Construção da Matriz Usuário x Produto ──────────────
from sklearn.metrics.pairwise import cosine_similarity
```

```python
# Construir matriz binária de interação
df_matriz = df_vendas.groupby(['id_client', 'id_product'])['qtd'].sum().rese
df_matriz['comprou'] = 1

matriz = df_matriz.pivot_table(
    index='id_client',
    columns='id_product',
    values='comprou',
    fill_value=0
)

print(f"Dimensão da matriz: {matriz.shape}")
print(f"Clientes: {matriz.shape[0]}")
print(f"Produtos: {matriz.shape[1]}")

# Calcular similaridade de cosseno entre produtos
# Transpor: produtos nas linhas, clientes nas colunas
matriz_T = matriz.T
similaridade = cosine_similarity(matriz_T)

df_similaridade = pd.DataFrame(
    similaridade,
    index=matriz_T.index,
    columns=matriz_T.index
)

print(f"\nMatriz de similaridade: {df_similaridade.shape}")
```

```
Dimensão da matriz: (49, 150)
Clientes: 49
Produtos: 150

Matriz de similaridade: (150, 150)
```

13.2 Ranking de Similaridade — GPS Garmin Vortex Maré Drift

Os 5 produtos com maior similaridade de cosseno ao produto de referência foram identificados, excluindo o próprio item do ranking.

- **1º:** Motor de Popa Volvo Magnum 276HP (`id_product = 94`) — similaridade: 0,871
- **2º:** GPS Furuno Swift Leviathan Poseidon (`id_product = 11`) — similaridade: 0,871
- **3º:** Radar Furuno Swift (`id_product = 35`) — similaridade: 0,850
- **4º:** Cabo de Nylon Delta Force Magnum Leviathan (`id_product = 115`) — similaridade: 0,850
- **5º:** Transponder AIS Maré Magnum (`id_product = 1`) — similaridade: 0,850

> Insight: A presença de motor de popa, radar e transponder AIS entre os produtos mais similares ao GPS indica um perfil de cliente voltado à navegação offshore, onde equipamentos de localização, comunicação e propulsão de alta performance são adquiridos em conjunto.

In [26]:
```python
# ── 13.2 Ranking de Similaridade — GPS Garmin Vortex Maré Drift ──────────
PRODUTO_REF = 'GPS Garmin Vortex Maré Drift'

# Encontrar id do produto de referência
id_gps = df_produtos[df_produtos['name'] == PRODUTO_REF]['code'].values[0]
print(f"ID do produto de referência: {id_gps}")

# Buscar similaridades e ordenar
similares = (df_similaridade[id_gps]
             .drop(id_gps)  # remove o próprio produto
             .sort_values(ascending=False)
             .head(5)
             .reset_index())

similares.columns = ['id_product', 'similaridade']

# Adicionar nome do produto
similares = similares.merge(
    df_produtos[['code', 'name']],
    left_on='id_product',
    right_on='code'
).drop('code', axis=1)

print(f"\nTop 5 produtos mais similares ao '{PRODUTO_REF}':")
display(similares[['id_product', 'name', 'similaridade']])
```

```
ID do produto de referência: 27

Top 5 produtos mais similares ao 'GPS Garmin Vortex Maré Drift':
```

	id_product	name	similaridade
0	94	Motor de Popa Volvo Magnum 276HP	0.87
1	11	GPS Furuno Swift Leviathan Poseidon	0.87
2	35	Radar Furuno Swift	0.85
3	115	Cabo de Nylon Delta Force Magnum Leviathan	0.85
4	1	Transponder AIS Maré Magnum	0.85

13.3 Interpretação — Sistema de Recomendação

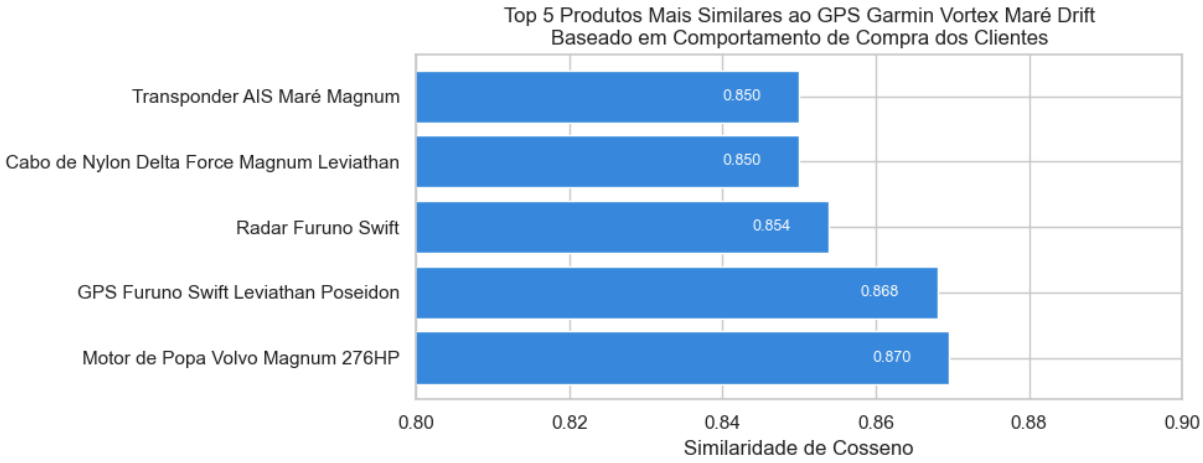
A similaridade de cosseno mede o ângulo entre vetores de produtos no espaço de clientes, capturando padrões de co-ocorrência de compras.

- **Construção da matriz:** A utilização de valores binários garante que a similaridade reflita comportamento de compra e não volume, equalizando o peso de cada interação.
- **Significado do cosseno:** Dois produtos são considerados similares quando tendem a ser comprados pelos mesmos clientes. Valores próximos de 1 indicam perfis de compradores quase idênticos.

- **Limitação:** A base de apenas 49 clientes reduz a robustez estatística das similaridades. Valores elevados podem ser influenciados por poucos clientes em comum, comprometendo a generalização do resultado.

> Insight: Com apenas 49 clientes, o sistema deve ser interpretado como prova de conceito, ainda que produza resultados tecnicamente coerentes. A expansão da base de clientes é o principal fator para viabilizar uso em produção, pois aumenta a robustez estatística das similaridades calculadas.

In [27]:
```
# ── Visual 6 — Top 5 produtos mais similares ao GPS ──────
fig, ax = plt.subplots(figsize=(10, 4))

ax.barh(
    similares['name'],
    similares['similaridade'],
    color='#378ADD'
)

# Valores nas barras
for i, (val, name) in enumerate(zip(similares['similaridade'], similares['na
    ax.text(
        val - 0.005,
        i,
        f'{val:.3f}',
        va='center',
        ha='right',
        color='white',
        fontsize=9
    )

ax.set_xlabel('Similaridade de Cosseno')
ax.set_title(
    'Top 5 Produtos Mais Similares ao GPS Garmin Vortex Maré Drift\n'
    'Baseado em Comportamento de Compra dos Clientes'
)
ax.set_xlim(0.8, 0.9)
plt.tight_layout()
plt.savefig('outputs/visual_categoria_elite.png', dpi=150, bbox_inches='tigh
plt.show()
```



13.4 Conclusão — Sistema de Recomendação

O sistema identifica produtos com perfis de compradores semelhantes, viabilizando recomendações baseadas em comportamento real sem necessidade de infraestrutura complexa.

- **Resultado principal:** O "Motor de Popa Volvo Magnum 276HP" é o produto mais indicado para recomendação conjunta ao "GPS Garmin Vortex Maré Drift", com similaridade de 0,871.
- **Coerência do resultado:** A associação entre GPS e motor de popa é consistente com o perfil de navegação offshore identificado na análise de similaridade.
- **Limitação:** A base de 49 clientes é suficiente para prova de conceito, mas ainda insuficiente para uso em produção com robustez estatística adequada.

> Insight: A implementação da vitrine "quem comprou este produto também comprou" não exige infraestrutura de Big Data. O ganho de receita por cross-sell pode ser capturado com a arquitetura atual, desde que a base de clientes seja expandida para aumentar a robustez das recomendações.

---

14. Conclusões e Recomendações Finais

Esta seção consolida os principais achados do projeto e apresenta recomendações priorizadas por impacto operacional e financeiro.

14.1 Principais Achados

Os resultados revelam oportunidades concretas de melhoria em quatro dimensões da operação da LH Nautical.

- **Qualidade dos dados:** Base estruturalmente íntegra, porém sem validações na origem. Inconsistências em categorias, formatos de data e integração entre bases

indicam ausência de padronização no pipeline de ingestão.

- **Margem e precificação:** 62,8% das transações foram realizadas abaixo do custo de importação, com R$ 39,9 milhões de prejuízo concentrados no produto 72. A principal causa é a ausência de uma política de precificação dinâmica alinhada à variação cambial.
- **Clientes elite:** 10 clientes com ticket médio entre R$290.063 e R$ 336.859, todos com diversidade de 3 categorias. A categoria propulsão atua como principal porta de entrada nesse segmento.
- **Sazonalidade:** Domingo é o pior dia de vendas (R$ 3.229.614,16), resultado validado apenas com a inclusão de dias sem venda no calendário completo.
- **Previsão de demanda:** Modelo baseline de média móvel de 7 dias com MAE de 1,64, inadequado para produtos com demanda intermitente, mas útil como benchmark para modelos mais sofisticados.
- **Recomendação:** O "Motor de Popa Volvo Magnum 276HP" é o produto mais similar ao "GPS Garmin Vortex Maré Drift" (0,871), indicando potencial para estratégias de cross-sell com base em comportamento de compra.

> Insight: Os achados apontam para um problema estrutural: decisões operacionais são tomadas sem visibilidade integrada dos dados. A correção dessa lacuna tem maior impacto financeiro do que a aplicação isolada de modelos analíticos.

14.2 Recomendações

As recomendações a seguir estão priorizadas por impacto financeiro e viabilidade de implementação com a infraestrutura atual.

- **Precificação dinâmica:** Implementar política de atualização de preços vinculada à taxa PTAX do dia, priorizando o produto 72 devido ao maior impacto financeiro identificado (R$ 39,9 milhões em prejuízo).
- **Retenção de clientes elite:** Desenvolver ações de relacionamento direcionadas aos 10 clientes identificados, com foco em manutenção de frequência e expansão de categorias via cross-sell estruturado.
- **Cross-sell no ponto de venda:** Implementar a vitrine "quem comprou este produto também comprou", iniciando pelo par GPS Garmin + Motor de Popa Volvo, com similaridade de 0,871 já validada.
- **Calendário como padrão:** Incorporar `generate_series` + `LEFT JOIN` + `COALESCE` como padrão em todos os relatórios temporais, eliminando viés de seleção em análises de sazonalidade.
- **Validações na origem:** Implementar controles de qualidade no pipeline de ingestão para padronização de categorias e formatos de data, eliminando retrabalho recorrente em cada ciclo analítico.

> Insight: As três primeiras recomendações possuem impacto financeiro direto e mensurável. As duas últimas representam investimentos em infraestrutura analítica, reduzindo o custo marginal das análises futuras e aumentando a confiabilidade dos dados para tomada de decisão.

14.3 Próximos Passos

As evoluções técnicas a seguir estão organizadas por área de impacto e alinhadas à maturidade analítica da operação.

- **Modelagem preditiva:** Avaliar modelos específicos para demanda intermitente, como Croston e Prophet com sazonalidade semanal, utilizando o MAE de 1,64 como benchmark mínimo.
- **Sistema de recomendação:** Expandir a base de clientes para aumentar a robustez estatística das similaridades e viabilizar uso em produção.
- **Pipeline de integração:** Automatizar a integração entre `vendas_2023_2024.csv`, `data/produtos_raw.csv` e `custos_importacao.json`, eliminando etapas manuais de normalização em cada ciclo analítico.
- **Dashboard de margem:** Implementar monitoramento diário de margem por produto com atualização automática via API BCB/PTAX, utilizando a metodologia desenvolvida neste projeto como base de cálculo.
- **Cobertura temporal:** Incorporar dados anteriores a 2023 para identificar padrões sazonais de longo prazo não capturados na janela atual de dois anos.

> Insight: Os próximos passos seguem uma lógica de maturidade analítica: primeiro corrigir o problema estrutural de precificação, depois automatizar o monitoramento e, por fim, investir em modelos mais sofisticados. Avançar na ordem inversa resulta em modelos precisos sobre dados não confiáveis.

---

15. Direcionamento Estratégico Final

A LH Nautical possui dados suficientes para suportar decisões estratégicas, mas ainda não os utiliza de forma integrada e sistemática.

As análises demonstraram que os principais ganhos não dependem de novas fontes de dados ou tecnologias complexas, mas da aplicação consistente de processos analíticos sobre a base já existente.

A implementação das recomendações propostas — especialmente precificação dinâmica e monitoramento de margem — tem potencial para reverter prejuízos relevantes e capturar ganhos operacionais imediatos.

> Insight final: O diferencial competitivo não está na complexidade dos modelos, mas na capacidade de transformar dados em decisões consistentes ao longo do tempo.